

	Pràctica Entregable 01 Classe per encriptar informació	IMO30FP01 REV 00 11/13
---	---	----------------------------------

CURS:	DAM2	DATA:	Setembre-2026	Qualificació	
MÒDUL:	M0490 - Programació Serveis i Processos				
TEMA:	Encriptació i desencriptació de dades				
RA	RA5				
Professors	Cristina Reina - Carles Bonet				

Enunciat:

Formeu part de l'equip de desenvolupament d'una empresa que necessita **enviar petits missatges protegits entre diferents aplicacions**.

L'empresa no vol que el programa principal conegui els detalls de com es protegeixen els missatges. Per aquest motiu, **us demana crear una classe criptogràfica independent i reutilitzable**.

La classe haurà de permetre:

MISSATGE + CLAU → Procés d'ENCRYPTACIÓ → MISSATGE ENCRYPTAT

MISSATGE ENCRYPTAT + CLAU → Procés de DESENCRIPTACIÓ → MISSATGE ORIGINAL

La vostra missió és **dissenyar el vostre propi sistema de xifrat** i implementar-lo com una classe reutilitzable.

Un exemple per entendre com crear el vostre propi sistema:

Suposem que tenim:

- Missatge: HOLA
- Clau: 3

Podem utilitzar la clau per desplaçar cada lletra tres posicions:

- H → K
- O → R
- L → O
- A → D

Per tant: HOLA → KROD

Per recuperar el missatge original, fem el procés invers: KROD → HOLA.

1. Investiga el funcionament

Respon aquestes preguntes:

- Què passa si encriptem dues vegades el mateix missatge amb la mateixa clau? ·

En el cas de encriptar amb el metode de desplaçament amb la mateixa clau lo unic que faria es que la clau sigues el doble de la clau original, per exemple si diem clau 2 i el mistatge es A, el resultat sera C, i com ho tornem a fer amb la mateixa clau pasaria de C a E, per lo que es lo mateix a fer clau 4, el cual es el doble de la clau original.

Què passa si canviem la clau? Prova HOLA + clau 3 i HOLA + clau 7.

En cas de canviar la clau trobarem que el resultat de la encriptacio es completament diferent, ja que dir una clau 1 a clau 2 fa que trobem que ningun resultat es el mateix. Amb clau tres quedaria KROD, amb clau 7 cada lletra es desplaça 4 vegades mes.

- Què passa si intentem desencriptar utilitzant una clau incorrecta?

Que el resultat al intentar desencriptar sera un missatge completament diferent al que es va encriptar

Com a mínim haurà de complir: **La clau**

- El mateix missatge amb la mateixa clau ha de produir el mateix resultat.
- Una clau diferent ha de modificar el resultat.
- Si utilitzem la clau correcta, hem de poder recuperar exactament el missatge original.

Desencripta(Encripta("HOLA", "123"), "123") = "HOLA"

2. Dissenya l'algoritme propi d'encriptació i desencriptació

Abans de programar, heu de dissenyar el vostre sistema de xifrat.

Podeu utilitzar, per exemple:

- un sistema de substitució.
- un sistema de desplaçament.
- una combinació de diferents operacions.

La clau ha de participar obligatòriament en el procés.

Nom del vostre sistema	
Tipus de sistema	☐ Substitució ☐ Desplaçament ☐ Combinació ☐ Altres: _-_____

Tipus de clau	
Com funciona l'enciptació?	Cada lletra del abecedari sera te un numero assignat, A= 1, B=2... Aixi una vegada rebem el missatge que volem encriptar el convertim a numero. Llavors una vegada ho tenim cada lletra convertida el posem a un arraylists. Despres multipliquem cada espai del arraylists per els decimals del numero pi, per exemple el primer numero del array per el primer decial del arraylists i seguim aixi. Evitarem els 0 del numero pi. Una vegada fet tindrem un mathrandom el cual es entre 1 y els characters que tenim en el missatge. Per exemple si el missatge es de 30 lletres seria ente 1 i 30. El resultat per exemple es 3, al ser tres vol dir que cada 3 vegades ens movem en el arraylists aquell numero el sera elevat a la clau. Una vegada a acabat tot el que farem es guradar el mathrandom al final del arraylists.
Com funciona la desenciptació?	Al desenciptar primer agafem el array i agafem el final del array per tenir el math random. Una vegafa fet agafem y fem l'arrel amb el numero del math per desfer allo, despres de allo agafem el numeros, els dividem entre el seu corresponent del numero pi y finalment pasem el numero a les lletres assignades.
Com sé que el meu sistema funciona?	Perque son operacions matematiques les quals es resolen fent al revés.

2

3. Demostra el teu algoritme

Abans de començar a programar, feu un exemple manual complet.

Missatge: HOLA

Clau: _____7_____

Enciptació: H → _8_ O → _15_ L → _12_ A → _1_

Resultat: __[24, 15, 48, 1]__

Igualment, desencipta el resultat i demostra que recuperes HOLA.

4. Implementació de la classe “Encripta” en Java

Crea una classe independent anomenada:

ClasseCriptografica

La classe haurà d'oferir, com a mínim, aquests dos mètodes:

- **Encripta(Missatge, Clau)**
- **Desencripta(Missatge, Clau)**

Els mètodes han de retornar el missatge resultant.

La classe criptogràfica no ha de contenir el programa principal. El programa principal serà qui utilitzarà la classe.

Programa principal → ClasseCriptografica → Encripta / Desencripta

5. Proves del sistema dissenyat d'encriptació

Test	Prova	Resultat
Test 1	Missatge curt: HOLA	Correcte Observacions: HOLA
Test 2	Missatge amb espais: HOLA MÓN	Incorrecte Observacions: HOLA MN
Test 3	Missatge més llarg: almenys 30 caràcters	Correcte Observacions: Funciona sense errors
Test 4	Clau diferent: comprovar que el resultat canvia	Correcte Observacions: El missatge encriptat canvia completament per el us que es dona a la clau

3

Test 5	Encriptar i desencriptar: recuperar exactament el missatge original	Correcte Observacions: Es recupera tot menys els accents i si es majúscula o minúscula
--------	--	--

Test 6	Clau incorrecta: explicar què passa	Correcte Observacions: Al donar una clau que no sigui un numero sencer donara error fent que no poguis seguir amb el missatge, ja que la clau a de ser un numero sencer.
--------	-------------------------------------	--

6. Implementació d'un sistema de xifrat AES en Java

Fins ara heu dissenyat i implementat el vostre propi sistema de xifrat. Ara fareu un pas més: **utilitzareu AES (Advanced Encryption Standard)**, un algoritme de xifrat simètric utilitzat en sistemes reals.

En aquesta part **no haureu de crear l'algoritme AES des de zero**. Java proporciona una API de criptografia que permet utilitzar-lo.

Objectiu

Crear una classe anomenada **ClasseAES** que permeti:

MISSATGE + CLAU

↓

AES

↓

MISSATGE XIFRAT

i recuperar posteriorment el missatge original:

MISSATGE XIFRAT + CLAU

↓

AES

↓

MISSATGE ORIGINAL

La classe haurà d'oferir, com a mínim, els dos mètodes següents:

```
public static String encripta(String missatge, String clau)
public static String desencripta(String missatgeXifrat, String clau)
```

6.1. Eines disponibles

Per realitzar aquesta activitat podeu utilitzar les classes i mètodes següents de

Java: **Classes:**

```
import javax.crypto.Cipher;  
import javax.crypto.spec.SecretKeySpec;  
import java.nio.charset.StandardCharsets;  
import java.util.Base64;
```

Mètodes i elements que podeu consultar i utilitzar:

```
getBytes()  
Cipher.getInstance()  
cipher.init()  
cipher.doFinal()  
Base64.getEncoder()  
Base64.getDecoder()  
SecretKeySpec  
Cipher.ENCRYPT_MODE  
Cipher.DECRYPT_MODE
```

Aquestes són les eines que necessitareu per construir el vostre sistema.

No es proporcionarà el codi complet de la classe. Haureu d'investigar el funcionament de les classes i mètodes proporcionats i decidir com combinar-los per construir els processos d'enciptació i desenciptació

Trobareu un [annex explicatiu](#) breu al final del document.

6.2. Implementa la classe **ClasseAES**

Crea una classe independent anomenada [ClasseAES](#), que tingui els mètodes:

encripta (missatge, clau)
desencripta (missatge, clau)

El mètode `encripta()` haurà de:

5

1. Rebre un missatge i una clau.
2. Preparar la clau perquè pugui ser utilitzada per AES.
3. Crear i configurar el sistema de xifrat.
4. Convertir el missatge al format necessari.
5. Xifrar el missatge.
6. Convertir el resultat en un format que es pugui retornar com a `String`.
7. Retornar el missatge xifrat.

El mètode `desencripta()` haurà de realitzar el procés invers:

1. Rebre el missatge xifrat i la clau.
2. Preparar la clau.
3. Crear i configurar el sistema de desxifrat.
4. Recuperar les dades xifrades.
5. Desxifrar-les.
6. Convertir el resultat novament a text.
7. Retornar el missatge original.

6.3. Programa principal

Crea un `ProgramaPrincipal` que utilitzi la classe `ClasseAES`. Utilitza inicialment les dades següents:

`Missatge: Aquest és un missatge secret.`

`Clau: 1234567890123456`

El programa haurà de:

1. Mostrar el missatge original.
2. Encriptar-lo utilitzant AES.
3. Mostrar el missatge xifrat.
4. Desencriptar el resultat utilitzant la mateixa clau.
5. Mostrar el missatge recuperat.

El resultat final ha de demostrar que:

```
Desencripta(Encripta(missatge, clau), clau) = missatge 6
```

6.4. Experimenta amb el teu programa

Una vegada el programa funcioni, realitza les proves següents:

Prova 1. Clau correcta

Comprova que el missatge original es recupera correctament.

Ho he comprovat i trobo que hem funciona perfectament.

```
Introdueix la clau: feruigbwreipjvabsdjpigbeiu
Introdueix el missatge: hola, vull comprar una quantes pomes
oHT7URbVGKHS5vRYaEGmuXUll6yqPLooMmIPrhjRSZSkF3opPae/TEF/oH8HnCp
hola, vull comprar una quantes pomes
```

Prova 2. Clau diferent

Intenta desencriptar el missatge utilitzant una clau diferent.

Al introduir una clau que no es la correcta ens retorna null, això es perquè es desxifrat falla.

```
Introdueix la clau: fr9eugweh9vphbsduigvdhpt
Introdueix el missatge: hola espero et trobis be
EkeL4qcJfKjKYUcxRo2puffeI+HJrp2hpnaJgOaKwRA=
null
```

Explica què passa.

Prova 3. Missatge diferent

Utilitza un altre missatge i comprova que el procés continua

funcionant

No ens dona la solució, ja que al ser un altre missatge hauria de

donar error, pero al tenir tot dins de un try catch no ho provoca.

```
Introdueix la clau: frenugovfndbvjkfdñbgjkpg
Introdueix el missatge: holaaaaa
J9YY1vrUjodONtRya/JMug==
jewfongofdln dolgbunfdgn
```

Prova 4. Clau de longitud incorrecta

Prova d'utilitzar una clau que no tingui la longitud adequada.

Observa què succeeix i explica el motiu.

Aixo no suceix, ja que hi han de maneres de fer-ho, jo he escollit la cual si es massa llarg talla la clau, i si es molt curta la omple amb bytes de 00.

6.5. Comprèn el codi

No n'hi ha prou amb aconseguir que el programa funcioni. Has de poder explicar què està fent.

Respon les preguntes següents:

1. Quina classe de Java s'encarrega de realitzar el xifrat?

La de Cipher

2. Per a què serveix `SecretKeySpec`?

Per a pasar la clau a un objecte del tipus clau, que es amb el que treballa Cipher

3. Què fa `Cipher.getInstance()`?

Fa la instancia del chiper, apart de dir en que mode oh fara, en aquest cas AES.

4. Quina diferència hi ha entre `ENCRYPT_MODE` i `DECRYPT_MODE`?

El encrypt serveix per dir a Cipher que farem la encriptacio per la clau, en el cas de decrypt es el contrari, li diem que fara la desencryptacio.

5. Per a què serveix `doFinal()`?

Es el que acaba de crear la encriptacio.

6. Per què hem de convertir el missatge a bytes?

Perque AES unicament funciona amb els bytes.

7. Per què utilitzem Base64?

Serveix per convertir el missatge en un String, ja que sense el Base64 no seria llegible

8. Quina mida té la clau que has utilitzat?

Hauria de ser de 16, pero hi ha un metode que fa que la propia clau sigui 16 digits.

9. Què significa que AES sigui un sistema de xifrat simètric?

Aixo vol dir que s'utilitza la mateixa clau per cifrar y per descifrar.

10. Per què necessitem la mateixa clau per recuperar el missatge?

Perque AES es un xifrat simetric, això vol dir que si li donem una clau diferent ens donara un error.

Finalment, respon:

Per què és preferible utilitzar un algoritme criptogràfic estàndard com AES en lloc de crear el nostre propi sistema de xifrat?

Crec que es preferible perquè al utilitzar diferents tipus de capes es mes segur, comparantlo amb el meu sistema es molt mes complex, ja que el meu es sempre el mateix amb un petit mathrandom que es guarda al final del array que es passa, per lo que rapidament una persona podria aprendre a desxifrar el meu sistema. En canvi aquest te varies capes que va canviant constantment, això ho fa molt mes segut i complexe.

Lliurament

Heu de lliurar en un repositori GitHub:

1. Sistema de xifrat propi

- `ClasseCriptografica.java`
- `ProgramaPrincipal.java`

2. Implementació AES

- `ClasseAES.java`
- `ProgramaPrincipalAES.java`

3. Documentació

- Document PDF amb les respostes i explicacions de la pràctica.

Cal lliurar aquesta documentació en un Repositori GitHub i que el compartiu amb la Cristina i en Carles.

Annex Explicatiu

Xifrat AES

El xifrat AES (Advanced Encryption Standard) va ser aprovat com estàndard de xifrat pel govern dels EUA després d'un procés de 5 anys. El xifrat va ser desenvolupat per dos criptòlegs belgues J.Daemen i V. Rijmen que el van enviar amb el nom de Rijndael.

Característiques:

- Xifrat de bloc: mida 128 bits.
- Clau de xifrat de 128,192 o 256 bits.

Actualment es considera un sistema de xifrat segur (la NSA exigeix clau de 192 o 256 bits per documents d'alta seguretat). És un algoritme ràpid, AES 256 és més ràpid que DES i per tant molt més ràpid que la solució 3DES.

Elements del xifrat:

- IV (Vector inicialització): És necessari per iniciar cada procés de xifrat. Té una mida de 16 bytes (1 bloc). El seu objectiu és evitar que dos blocs iguals tinguin el

mateix criptograma.

- **Clau (Key):** És la principal informació per xifrar i desxifrar en els algoritmes simètrics. A la implementació per defecte de .NET s'utilitza la de 256 bytes. **Cipher mode:**

ECB: xifra cada bloc per separat. Per tant, blocs idèntics donen el mateix criptograma.

CBC: s'utilitza un vector d'inicialització (IV) i cada bloc es combina amb el bloc anterior amb una operació XOR. Això assegura que blocs iguals no donin un criptograma idèntic. **Clau de xifrat:**

Existeixen mètodes per obtenir la clau de forma automàtica, però això només té sentit si aquesta clau es pot enviar de forma segura. Volem obtenir la clau a partir de passphrase que introdueixi l'usuari.

La forma més còmoda per l'usuari és introduir una password o passphrase enlloc de una clau i un vector d'inicialització. Utilitzem `Rfc2898DeriveBytes` per obtenir un *hash* de la combinació dels dos textos. S'afegeix una cadena *salt* per fer més difícil l'atac per Rainbow Tables. L'algoritme es pot aplicar iterativament per augmentar la complexitat.

9

Rúbrica:

Criteri	Què s'avalua	Punts	
1. Disseny del sistema propi	Defineix l'algoritme, la clau i explica correctament encriptació i desencriptació	1,5	
2. Implementació del sistema propi	ClasseCRIPTografica funciona correctament i està separada del programa principal	1,5	
3. Ús de la clau	La clau participa realment en el procés i una clau incorrecta no permet recuperar correctament el missatge	1,0	
4. Proves del sistema propi	Realitza i documenta correctament els tests proposats	1,0	
5. Implementació d'AES	Utilitza correctament l'API de Java i implementa ClasseAES amb encriptació i	2,0	

	desencriptació		
6. Comprensió de l'API criptogràfica	Explica Cipher, SecretKeySpec, init(), doFinal(), Base64, etc.	1,0	
7. Experimentació amb AES	Realitza les proves amb diferents claus i missatges i explica els resultats	0,75	
8. Documentació i comparació	Respon les preguntes i compara el sistema propi amb AES de manera raonada	0,75	
9. Organització i lliurament	Estructura correcta del projecte, noms de classes, repositori GitHub i documentació	0,5	
TOTAL		10	